Notes

Data Modelling, Database, Data Warehouse & SQL

Dheeraj S Kulkarni
2025

Data Modeling Concepts

Data modelling is the process of creating a conceptual, logical, and physical representation of data in a system. It defines how data is structured, stored, and accessed, and it forms the foundation for databases and data warehouses.

---

1. Stages of Data Models

a. Conceptual Data Model

- **Purpose**: Provides a high-level view of the system, focusing on business requirements rather than technical details.
- **Key Features**:
  - Defines entities, relationships, and attributes.
  - No primary/foreign keys or constraints.
- **Example**:
  - Entities: Customer, Order
  - Relationship: Customer places Order

b. Logical Data Model

- **Purpose**: Specifies the structure of data in detail, including relationships, attributes, and constraints, but is independent of the database technology.
- **Key Features**:
  - Defines primary keys, foreign keys, and data types.
  - Still independent of physical storage.
- **Example**:
  - Customer (CustomerID, Name, Email)
  - Order (OrderID, OrderDate, CustomerID)

c. Physical Data Model

- **Purpose**: Maps the logical model to the actual database implementation.
- **Key Features**:
  - Includes storage-specific details like indexing, partitions, and physical constraints.
  - Optimized for performance.
- **Example**:
  - SQL table definitions with data types, constraints, and indexes.

---

2. Core Data Modeling Concepts

a. Entities and Attributes

- **Entities**: Objects or concepts in the real world (e.g., Customer, Product).
- **Attributes**: Characteristics of entities (e.g., Customer has Name, Email).

b. Relationships

- Define how entities are related to each other.
- **Types**:
  - One-to-One (1:1): A Person has one Passport.
  - One-to-Many (1:N): A Customer places multiple Orders.
  - Many-to-Many (M:N): Students enroll in multiple Courses.

c. Keys

- **Primary Key (PK)**: Unique identifier for a record (e.g., CustomerID).
- **Foreign Key (FK)**: Attribute in one table referencing the primary key of another table.

d. Normalization

- The process of organizing data to reduce redundancy and improve consistency.
- **Forms**:
  - **1NF**: Eliminate duplicate columns and ensure atomicity.
  - **2NF**: Ensure all attributes depend on the whole primary key.
  - **3NF**: Eliminate transitive dependencies.

e. Denormalization

- Combining tables or reducing normalization for performance optimization.
- Used in data warehouses to simplify querying.

---

3. Data Modeling in Data Warehouses

a. Star Schema

- **Structure**:
  - Central **fact table** surrounded by **dimension tables**.
- **Advantages**:
  - Simple and intuitive.
  - Optimized for OLAP queries.
- **Example**:
  - Fact Table: Sales (SaleID, ProductID, Amount, DateID)
  - Dimensions: Product, Time, Customer, Region

b. Snowflake Schema

- **Structure**:
  - Dimensions are normalized into multiple related tables.
- **Advantages**:
  - Reduces redundancy and saves storage.
- **Disadvantages**:
  - More complex queries.
- **Example**:
  - A Product dimension is split into Product and Category.

c. Fact Tables

- **Purpose**: Store numeric measures (e.g., Revenue, Quantity).
- **Types**:
  - **Transactional Facts**: Individual transactions.
  - **Snapshot Facts**: Periodic snapshots (e.g., daily inventory).

- **Accumulating Facts**: Show lifecycle data (e.g., order processing).

d. Dimension Tables

- **Purpose**: Provide descriptive context to facts.
- **Example**:
  - Time: Date, Week, Quarter, Year
  - Customer: Name, Age, Region

e. Slowly Changing Dimensions (SCD)

- Handle historical changes in dimension data.
- **Types**: SCD Types 0, 1, 2, 3, 4, 6.

---

4. Advanced Data Modeling Concepts

a. Hierarchies

- Define relationships within dimensions for drill-down/roll-up analysis.
- **Example**:
  - Geography: Country > State > City
  - Time: Year > Quarter > Month > Day

b. Surrogate Keys

- Synthetic primary keys are used in place of natural keys to ensure uniqueness and improve performance.
- **Example**:
  - Surrogate key CustomerSK instead of natural key CustomerID.

c. Bridge Tables

- Used to resolve many-to-many relationships.
- **Example**:
  - A StudentCourse bridge table linking Students and Courses.

d. Metadata

- Data about the data model, providing context and rules for usage.
- Types:
  - **Technical Metadata**: Table structures, indexes, constraints.
  - **Business Metadata**: Descriptions of business terms and metrics.

Summary Table of Dimension Types:

Dimension Type	Key Use Case
Conformed Dimension	Shared consistency across fact tables
Slowly Changing Dimension	Capturing changes over time
Junk Dimension	Combining unrelated low-cardinality flags
Role-Playing Dimension	Reusing dimensions in multiple contexts
Degenerate Dimension	Tracking keys without descriptive attributes
Inferred Dimension	Placeholder records for incomplete data
Shrunken Dimension	Aggregation at summary levels
Hierarchical Dimension	Enabling drill-down/roll-up analysis

Slowly Changing Dimension (SCD)

- **Definition**: Dimensions that capture changes in data over time.
- **Types**:
  - **Type 0**: Fixed values.
  - **Type 1**: Overwrite changes.
  - **Type 2**: Add a new row for changes.
  - **Type 3**: Add a new column for changes.
  - **Type 4**: Use a historical table.
  - **Type 6**: Hybrid approach.

Choosing an SCD Type

- **Type 0**: When data must never change (e.g., original launch date).
- **Type 1**: When historical data isn't necessary (e.g., email addresses).
- **Type 2**: When complete history is essential (e.g., address changes).
- **Type 3**: When only the most recent change is relevant (e.g., product reclassification).
- **Type 4**: When separating current and historical data is practical (e.g., employee role history).
- **Type 6**: When combining flexibility and history is required (e.g., customer loyalty programs).

How Data Warehouse Differs from a Database

Feature	Database	Data Warehouse
Purpose	Designed for day-to-day operations (OLTP).	Optimized for analytics and reporting (OLAP).
Data Type	Current, transactional data.	Historical, aggregated data.
Schema	Highly normalized (to reduce redundancy).	Denormalized (to optimize query performance).
Read/Write Operations	Frequent inserts/updates/deletes.	Primarily read operations with periodic data loads.
Data Integration	Data from one system (or a few).	Consolidates data from multiple systems.
Performance	Optimized for fast transactions.	Optimized for complex queries and large-scale analytics.
Users	Operational users (e.g., sales clerks).	Analytical users (e.g., business analysts).
Updates	Real-time updates.	Periodic updates (batch or ETL processes).
Indexing	Primary and foreign keys.	Star or snowflake schema indexing for analytical queries.

Kimball Data Modeling vs Relational Data Modeling:

Aspect	Kimball Data Modeling	Relational Data Modeling
Purpose	Designed for data warehousing and analytics (OLAP).	Designed for transactional systems (OLTP).
Focus	Optimizes data for querying and reporting.	Ensures data integrity and minimizes redundancy.
Approach	Dimensional modelling with fact and dimension tables.	Entity-relationship (ER) modelling with entities and relationships.
Schema Type	Star schema or Snowflake schema.	Fully normalized schema (up to 3NF or higher).
Normalization	Denormalized for performance and simplicity.	Highly normalized to reduce redundancy.
Data Granularity	Stores aggregated and historical data.	Stores detailed, current, and operational data.
Scalability	Suited for analytical scalability and large datasets.	Scales well for transactional operations.
Fact Tables	Central table with measures (e.g., sales, revenue).	Not applicable.
Dimension Tables	Descriptive attributes (e.g., product, time, customer).	Not applicable.
Key Use Cases	Dashboards, reporting, and business intelligence.	Transactional systems and operational applications.

Design Philosophy	Data is organized for readability and reporting.	Data is organized for storage efficiency and integrity.
Example	- **Fact Table**: `Sales` with `SalesAmount`, `ProductID`, `DateID`. - **Dimension Table**: `Product` with `ProductName`, `Category`.	- **Table 1**: `Customer` with `CustomerID`, `Name`, `Email`. - **Table 2**: `Order` with `OrderID`, `CustomerID`, `OrderDate`.
When to Use	For analytical systems, data warehouses, and historical analysis.	For operational systems, applications, and transaction processing.

Types of Approaches in the Dimensional Data Modelling

1. Bottom-Up Approach (Kimball Methodology)

- **Description**: Focuses on creating **data marts** first for specific business processes (e.g., sales, marketing) and then integrating them into a data warehouse.
- **Characteristics**:
  - Prioritizes business needs and end-user reporting requirements.
  - Uses **star schema** or **snowflake schema**.
  - Scalable by adding data marts as needed.
- **Use Case**: Smaller organizations or teams focusing on immediate reporting needs.

---

2. Top-Down Approach (Inmon Methodology)

- **Description**: Starts with building a **centralized enterprise data warehouse** that stores normalized data, and then data marts are created for specific reporting purposes.
- **Characteristics**:
  - Enterprise-focused with a highly normalized data warehouse (3NF).
  - Data marts are derived from the central repository.
  - Emphasizes data consistency and integration.
- **Use Case**: Large organizations require a single source of truth for analytics.

---

3. Hybrid Approach

- **Description**: Combines the strengths of both the bottom-up and top-down approaches.
- **Characteristics**:
  - Develops a **centralized data warehouse** (top-down) while also building **data marts** (bottom-up) for quick wins.
  - Balances enterprise-wide consistency with flexibility for departmental needs.
- **Use Case**: Organizations aiming for scalability while addressing immediate reporting needs.

Order of Execution of an SQL Query

1. `FROM` Clause
2. `WHERE` Clause
3. `GROUP BY` Clause
4. `HAVING` Clause
5. `SELECT` Clause
6. `ORDER BY` Clause
7. `LIMIT` Clause

How to Optimize SQL Queries Using the Order of Execution

- Use only the required tables.
- Avoid excessive joins and subqueries unless necessary.
- Filter data as early as possible to reduce the dataset size.
- Avoid Functions on Indexed Columns.
- Minimize the number of grouped columns.
- Use columns with low cardinality for grouping.
- Push as many filters as possible to the `WHERE` clause instead of `HAVING` to reduce intermediate data.
- Select only the required columns. Avoid `SELECT *`
- Use `LIMIT` to reduce the number of rows sorted if applicable.
- Use tools like **EXPLAIN** or **EXPLAIN ANALYZE** to check query execution plans and identify bottlenecks like table scans or improper index usage.
- Use `DISTINCT` judiciously. Ensure duplicate data is minimized at the source
- Precompute and store results for frequently used queries to avoid recomputation.
- Use `UNION ALL` instead of `UNION` if duplicate rows are not an issue, as `UNION` performs an extra step to remove duplicates.

Key Phrases for Interviews

- "I ensure indexing on columns used in JOINs, WHERE, and ORDER BY clauses."
- "I analyze query execution plans to identify bottlenecks."
- "I prioritize filtering data early in the WHERE clause to reduce processing overhead."
- "I avoid correlated subqueries and instead use joins or CTEs."
- "I optimize UNION queries by using UNION ALL where duplicates aren't a concern."

AWS vs Azure Services

Here's a comparison table of **AWS and Azure services** that serve similar purposes:

Category	AWS Service	Azure Service
Compute	Amazon EC2	Azure Virtual Machines (VMs)
Container Orchestration	Amazon Elastic Kubernetes Service (EKS)	Azure Kubernetes Service (AKS)
Serverless	AWS Lambda	Azure Functions
Storage	Amazon S3	Azure Blob Storage
File Storage	Amazon Elastic File System (EFS)	Azure Files
Block Storage	Amazon Elastic Block Store (EBS)	Azure Disk Storage
Networking	Amazon VPC	Azure Virtual Network (VNet)
Load Balancer	Elastic Load Balancing (ELB)	Azure Load Balancer
Content Delivery	Amazon CloudFront	Azure Content Delivery Network (CDN)
DNS Management	Amazon Route 53	Azure DNS
Databases	Amazon RDS	Azure SQL Database
	Amazon DynamoDB	Azure Cosmos DB
Data Warehouse	Amazon Redshift	Azure Synapse Analytics
Big Data	Amazon EMR	Azure HDInsight
Identity	AWS IAM	Azure Active Directory (AAD)
Monitoring	Amazon CloudWatch	Azure Monitor
IoT	AWS IoT Core	Azure IoT Hub
AI/ML	Amazon SageMaker	Azure Machine Learning

Queue Service	Amazon SQS	Azure Queue Storage
Pub/Sub	Amazon SNS	Azure Service Bus
Secrets Management	AWS Secrets Manager	Azure Key Vault
DevOps	AWS CodePipeline	Azure DevOps
Migration	AWS Migration Hub	Azure Migrate
Hybrid Cloud	AWS Outposts	Azure Stack
Backup	AWS Backup	Azure Backup